Intro To ML
Prof. Mathieu Salzmann

Luigi Ieracitano

February 25, 2026

Contents

1 Introduction **2**

1.1 What is Machine Learning ? . . . 2

1.1.1 Data . . . 2

1.1.2 Insight (what we discover from the datas) . . . 2

1.1.3 Algorithms . . . 3

2 Supervised Linear Model **4**

2.1 Notation . . . 4

2.2 Linear Regression . . . 4

2.2.1 Training . . . 4

2.2.2 Prediction . . . 5

2.2.3 1D linear regression . . . 5

2.2.4 Two input dimensions . . . 6

2.2.5 Hyperplane . . . 6

2.2.6 Minimizing multivariate function . . . 6

2.2.7 Solution of the linear regression . . . 6

2.2.8 Test time . . . 8

2.3 Model evaluation . . . 8

2.3.1 Evaluation metrics for regression . . . 8

2.3.2 Linear regression with multiple output dimensions . . . 9

2.3.3 Model . . . 9

2.4 Linear classification . . . 10

Introduction

1.1 What is Machine Learning ?

Machine learning = Data →Algorithms →Insight

1.1.1 Data

Data = attributes/tabular, text, speech, images, mixed.

Definition 1 (Data sample)**.** An individual observation. E.g., an image, or a list of attributes for one patient

Definition 2 (Data set)**.** A collection of multiple data sample. E.g., a collection of images, the attribute lists for multiple patients.

Unsupervised data : Each sample consists only of an observation, e.g., an image (without information about its content).
Supervised data : Each sample comes with additional annotations, e.g., category label.

1.1.2 Insight (what we discover from the datas)

What insight ?

- Precise, concrete prediction. E.g., the category depicted by an image.
- Better understanding of a phenomenon. E.g., identify the mother's charasteristics that lead to low birth weight.
- Better understanding of data. E.g., analyze the influences between different musical genres.
- **in this course** we mostly focus on precise insights. In particular we will study two classes of ML problems : **regression** and **classification**

Regression : Predict continuous value(s) for a given sample. E.g., predict the birth weight of a baby, human pose estimation : predict the 3D positions of human joints (multiple values).
Classification : Predict one discrete label for a given sample. E.g., binary classifications for movies (like vs not like; 1 or 0), muti-class image recognition (cat vs tree vs car; 0 vs 1 vs 2).

Order:
In regression, the values typically follow an order. E.g, predicting a weight of 3002g insted of 2977g is better that predicting 2500g.
In classification, the categories do not follow an order. E.g, predicting a "tree" instead of a "cat" is just as wrong as predicting "car". →here it's true or false.

1.1.3 Algorithms

What (classes of) algorithms ?

- Supervised learning :
  - Relies on supervised data
  - The annotations typically correspond to the desired insight
- Unsupervised learning :
  - Relies on unsupervised data
  - The goal is rather to analyze the observed data set

Unsupervised learning:

- Single stage: Analyze the whole data or transform it for further analysis.

Supervised learning:

- Stage 1: Training: Use data with ground-truth (info that is true) labels to optimize model parameters.
- Stage 2: Testing: Predict the output for a new data sample.
- Assumption : The annotations (correct responses) are the insight that we seek to obtain from the data. E.g.,: category label, human pose

Training set vs test set:

- The training set and the test set should always be **completely separate**.
- Never use the test annotations during training →the test set is not to train or to improve the performance of the model, but only to test his efficiency.
- Assumption: they are both drawn from the same statistical distribution. E.g., digit recognition: A model trained on MNIST will work poorly on Street View House Numbers

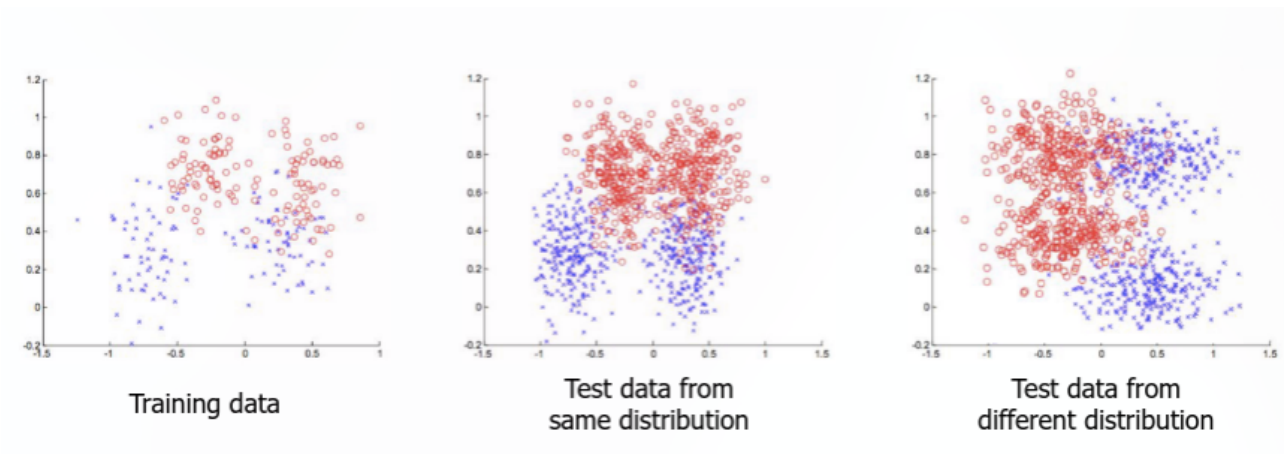


Figure 1.1: Same distribution assumption

Supervised Linear Model

2.1 Notation

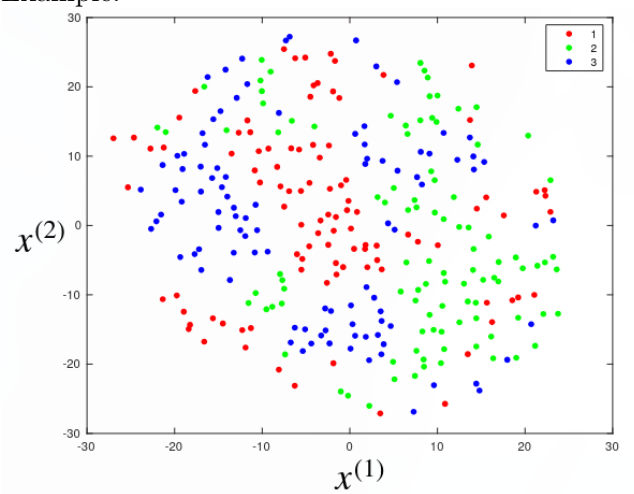
We denote the ith data sample (input) in the collection of N samples as

$$x_i \in \mathbb{R}^D,$$

a column vector of dimension D.
We denote the ith label (output) in the collection of N samples as y_i.
y_i :

- for classification represents the category the sample belongs to.
- for regression, it can be a single continuous value, or a column vector of dim C.

Example:



Here, x_i is one 2D point and thus D = 2, y_i is a discrete value indicating the class (color), i.e, 1, 2 or 3.

2.2 Linear Regression

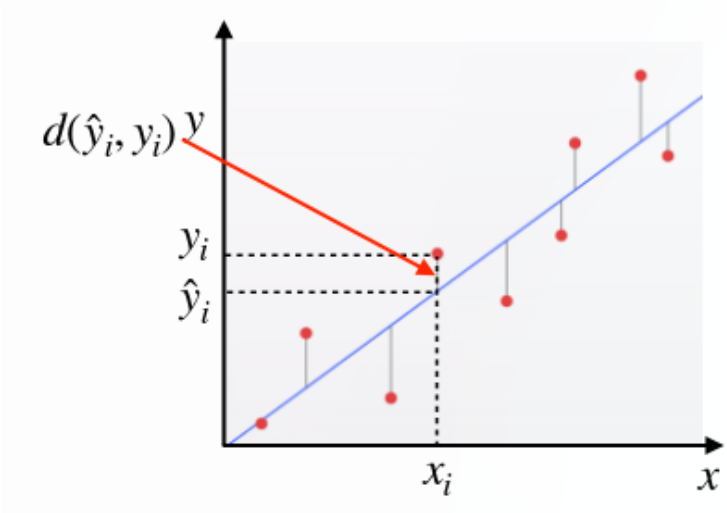
2.2.1 Training

Given N annotated data samples (x_i, y_i) with measurements, we can plot the data, and we will approximate it with a line. Our goal is to find the line that bests fits these measurements →linear regression. It consists of finding the best line parameters $w^{(0)*}$ and $w^{(1)*}$ for some given data.
This corresponds to the training stage:
Given N training pairs (x_i, y_i) , we aim to find $\left(w^{(0)}, w^{(1)}\right)$, s.t the predictions of the model

$$\hat{y}_i = w^{(0)} + w^{(1)} x_i$$

are close to the true values y_i.
The mesure of closeness/error between y_i and $\hat{y}_i$ is represented by the Euclidean distance:

$$d\left(\hat{y}_i, y_i\right) = \sqrt{\left(\hat{y}_i - y_i\right)^2}$$



$\hat{y}_i - y_i$: often reffered to as the *residual*

In practice, one often prefers using the squared Euclidian distance

$$d^2\left(\hat{y}_i, y_i\right) = \left(\hat{y}_i - y_i\right)^2$$

Training can then be expressed as the least-squared minization problem

$$min_{w^{(0)},w^{(1)}} \frac{1}{N} \sum_{i=1}^{N} d^2(\hat{y}_i, y_i)$$

2.2.2 Prediction

Once the best line for the given N observations is found, we can use it to predict a y value for a new x. Let $w^{(0)*}$ and $w^{(1)*}$ be the best line parameters given the observations.
Then

$$\hat{y} = w^{(0)*} + w^{(1)*}x$$

is the estimate value of y for any x.

Lecture 2 — 2026-02-24: Linear Regression (continuous insight)

2.2.3 1D linear regression

With 1D inputs, training a linear regressor aims to minimize

$$R\left(w^{(0)}, w^{(1)}\right) = \frac{1}{N} \sum_{i=1}^{N} d^2\left(\hat{y}_i, y_i\right) = \frac{1}{N} \sum_{i=1}^{N} \left(w^{(0)} + w^{(1)}x_i - y_i\right)^2$$

Equivalently, we can write

$$R\left(w\right) = \frac{1}{N} \sum_{i=1}^{N} \left(w^T x_i - y_i\right)^2 = \frac{1}{N} \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2$$

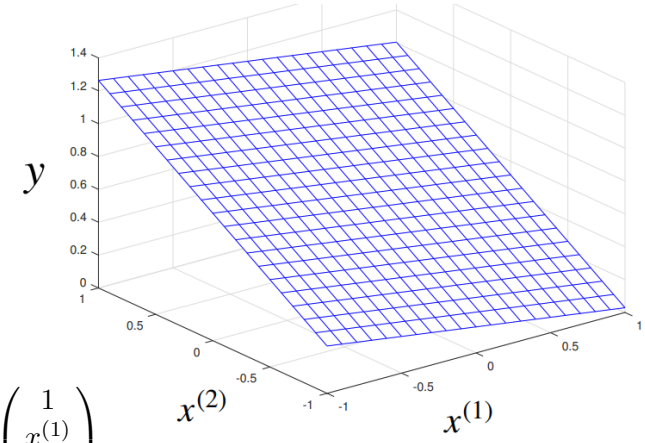
where

$$x_i = \begin{pmatrix} 1 \\ x_i \end{pmatrix} \in \mathbb{R}^2 \qquad w = \begin{pmatrix} w^{(0)} \\ w^{(1)} \end{pmatrix} \in \mathbb{R}^2$$

2.2.4 Two input dimensions

Here instead of a line, we can define a plane.
Expressed

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \end{pmatrix}$$



2.2.5 Hyperplane

This can be generalized to higher dimensions.
In dimension D we can write

$$y = w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + \ldots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Whatever the dimension, we can write

$$y = w^T x \quad x \in \mathbb{R}^{D+1}$$

, where the extra dimesion contains a 1 to account for $w^{(0)}$

Because the output remains 1D, we can still use the same least square loss function, and write training as

$$min_w \frac{1}{N} \sum_{i=1}^{N} d^2 \left(\hat{y}_i, y_i\right) \iff min_w \frac{1}{N} \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2$$

Solving this minimization problem involves computing its derivatives (derivation and gradient) w.r.t the model parameters (w).

2.2.6 Minimizing multivariate function

As in the 1D case (derivative = 0), to minimize a function, we seek the point w^* where the gradient where the gradient vanishes, i.e,

$$\nabla_w R(w^*) = 0$$

2.2.7 Solution of the linear regression

With our least-square formulation, we seek to minimize

$$R(w) = \frac{1}{N} \sum_{i=1}^{N} (x_i^T w - y_i)^2$$

This is a convex function of **w**, which means that we search for parameters w^* s.t

$$\nabla_w R(w^*) = 0$$

Let us start by computing the gradient $\nabla_w R(w)$

Hint 1: $R(w)$ is an average over the N training samples

- As the gradient of an average is the average of the gradients, you can focus on computing the gradient

$$\nabla_w (x_i^T w - y_i)^2$$

Hint 2: Use the chain rule

- The function $(x_i^T w - y_i)^2$ has the form $f(g(w))$, with $f(g) = g^2$
- You can compute its gradient as $\frac{df(g)}{dg} \nabla_w g(w)$

Hint 3: To compute the gradient of g(w), you can make use of the following general vector derivative rule:

- Given 2 vectors (a,b) in $\mathbb{R}^D$

$$\nabla_b (a^T b) = a$$

So with the chain rule:

$$\nabla_w R = \frac{1}{N} \sum_{i=1}^{N} \frac{d(x_i^T w - y_i)^2}{d(x_i^T w - y_i)} \cdot \nabla_w (x_i^T w - y_i) = \frac{2}{N} \sum_{i=1}^{N} (x_i^T w - y_i) \cdot x_i$$

To obtain the solution, we search for w^* s.t

$$\nabla_w R(w^*) = \frac{2}{N} \sum_{i=1}^{N} x_i (x_i^T w^* - y_i) = 0$$

This means

$$(\sum_{i=1}^{N} x_i x_i^T) w^* = \sum_{i=1}^{N} x_i y_i$$

$(D+1) \times (D+1)$ matrix * $(D+1) \times 1$ vector = $(D+1) \times 1$ vector

Let us now group all x_i and all y_i in a matrix and a vector

$$x = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(D)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(D)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_N^{(1)} & x_N^{(2)} & \dots & x_N^{(D)} \end{pmatrix} \in \mathbb{R}^{N \times (D+1)}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

Then we can re-write the solution as

$$X^T X w^* = X^T y$$

This finally give us

$$w^* = (X^T X)^{-1} X^T y = X^\dagger y$$

where $X^\dagger$ is known as the Moore-Penrose pseudo-inverse of X.

2.2.8 Test time

So now the predicted value $\hat{y}_t$ becomes

$$\hat{y}_t = \begin{pmatrix} 1 \\ x_t \end{pmatrix}^T w^* = (w^*)^T \begin{pmatrix} 1 \\ x_t \end{pmatrix}$$

2.3 Model evaluation

Once an ML model is trained, one would typically understand how well it performs on unseen test data

- At this stage, the parameters of the model are fixed
- Recall that the training and testing data must be separated!

During this evaluation, one compares the predictions of the model with the true annotations of the test data

- In contrast to the training stage, the model parameters are not updated
- The evaluation metric may directly be the loss function, but may also differ from it

2.3.1 Evaluation metrics for regression

Mean Squared Error (MSE): Same as the loss function but for N_t test samples.

$$MSE = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y}_i - y_i)^2$$

where $\hat{y}_i$ is the prediction for the test sample i and y_i the corresponding ground-truth value

Root Mean Squared Error (RMSE): Square-root of the MSE
Mean Absolute Error (MAE)

$$\frac{1}{N_t} \sum_{i=1}^{N_t} |\hat{y}_i - y_i|$$

Mean Absolute Percentage Error (MAPE)

$$MAPE = \frac{1}{N_t} \sum_{i=1}^{N_t} \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

Example, UCI Wine Quality dataset, predict the quality of wine based on several attributes

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Here the final RMSE is 0.65 for the training data and 0.63 for the test data

One can the look at the coefficient values $w^{(i)}$ to see the influence of each attribute
Warning: The magnitude of a coefficient will depend on the magnitude of the corresponding feature/attribute (A coefficient might be very small simply to compensate for the fact that the range of the feature is very large)

	Coeffecient
fixed acidity	0.017737
volatile acidity	-0.992560
citric acid	-0.139629
chlorides	-1.590943
free sulfur dioxide	0.005597
total sulfur dioxide	-0.003520
density	0.768590
pH	-0.437414
sulphates	0.812888
alcohol	0.301484

2.3.2 Linear regression with multiple output dimensions

Example of multiple output: human pose estimation

2.3.3 Model

The linear model has to be modified to output multiple values (not a vector w anymore because $w^T x_i$ output a single value)
So we use a matrix $\mathbf{W} \in \mathbb{R}^{(D+1)\times C}$ s.t

$$\hat{y}_i = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix} x_i$$

where each $w_{(j)}$ is a $(D+1)$-dimensional vector, used to predict one output dimension.
The multi-output linear regression is written

$$min_w \sum_{i=1}^{N} \left\| W^T x_i - y_i \right\|^2$$

where $\|a\| = \sqrt{\sum_{i=1}^{C} (a^{(j)})^2}$ indicates the norm of a vector (here a $\in \mathbb{R}^C$

To compute the gradient of this function, we make use of the following properties:
1. Norm and derivative:

$$\|a\|^2 = a^T a$$

and thus

$$\nabla_a \|a\|^2 = 2a$$

2. Matrix derivative:

$$\nabla_B (a^T B) = a$$

Thus using the chain rule we obtain the gradient

$$\nabla_W R = 2 \sum_{i=1}^{N} x_i (x_i^T W - y_i^T)$$

setting it to zero means that

$$(\sum_{i=1}^{N} x_i x_i^T) W^* = \sum_{i=1}^{N} x_i y_i^T$$

$(D+1) \times (D+1)$ matrix * $(D+1) \times C$ matrix = $(D+1) \times C$ matrix
We can again group the inputs in a matrix $\mathbf{X} \in \mathbb{R}^{N\times(D+1)}$, but the outputs now need to be grouped in a matrix (not a vector anymore), as

$$Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \vdots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{pmatrix} \in \mathbb{R}^{N\times C}$$

Then, following the same strategy as before, we have

$$W^* = (X^T X)^{-1} X^T Y = X^\dagger Y$$

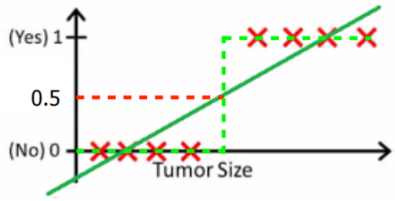
2.4 Linear classification

To predict one discrete label for a given sample e.g., binary classification or movies (like vs not), multi-class image recognition.
Classification can be tackle as a regression task (given a threshold we can classify the output in categories).

E.g. Classify a tumor as benign vs malignant

- 1D input (tumor size), 1D output (malignant/benign)
- Using a linear model, we can predict the label $\hat{y} = w^{(0)} + w^{(1)} x$

$$\text{label} = \begin{cases} 1 & \text{if } \hat{y}_t \geq 0.5 \\ 0 & \text{otherwise} \end{cases} \qquad \text{with } \hat{y}_t = \mathbf{x}_t^T \mathbf{w}^*$$



To fit the linear model to training data, the same strategy as for linear regression can be used:

- Given N pairs (x_i, y_i) , we can use the least-square loss to write

$$min_w \sum_{i=1}^{N} (x_i^T w - y_i)^2$$

So because the formulation is the same, the solution too

$$w^* = (X^T X)^{-1} X^T y$$